

XS standards

1. Variables
2. Code formatting
3. Files
4. Preprocessor features
5. Rules
6. Classes
7. Notes about optimization
8. Misc

1. Variables

- Class names start with uppercase letter or letters.
 - Class member variables are preceded with “m” (NOT “m_”). Exceptions are allowed for good reasons, such as contradicting standard mathematical conventions (a la `vec3.x / y / z`).
 - Underscores are only used when naming pre-processor directives, `#defines`, and macros.
 - Macros and `#defines` are named using all uppercase letters and underscores.
 - Hungarian notation is not used, it’s considered overkill, however...
 - Type prefixes such as “refFoo” are allowed if they seem appropriate
 - Local variables are always lowercase first. E.g. `int thisIsMyLocalVar`;
 - Declare function variables as needed in a function. Put the variable declarations next to where the variable is used (not all at the top of the function).
 - Global variables are preceded with “g”.
-

2. Code formatting

- Prefer `(foo == 0)` to `(0 == foo)`.
- **Indentation are three spaces with spaces replacing tabs.**
- Max line width of 130 characters. This is because otherwise the lines get too broad for compare software to show diffs without horizontally scrolling.
- When comparing something, always compare it against something else.
Example: `if (isFunctionTrue() == true)` or `if (isFunctionTrue() == false)`
NOT `if (isFunctionTrue())` or `if (!isFunctionTrue())`. The `!` can be easy to overlook and can lead to bugs.
- Curly braces are aligned under the leading character of the declaration of scope that they encapsulate. Code within that scope block is indented three spaces.
- All code within the same scope is indented to the same level.
- Function prototypes include all parameter names.
- Function parameters that cross multiple lines (e.g. in a declaration) have the secondary lines indented.
- Variables and function declarations within each section (public, protected, and private) of a class declaration are grouped. Each group is indented so that the variable declarations and function definitions line up.
- Switch statements are formatted as follows:

```
switch (i)
{
    case cFoo1:
```

```

{
    long b=2;
    foobar(b);
    break;
}
case cFoo2:
{
    foobar(1);
    break;
}
}

```

- Exceptions for switch statements are allowed if the data represents a table lookup, or for readability.
- ALL conditional statements and loops require brackets, even if they are one line calls - and on a separate line.

NOT

```

if (blah == true)
    return;

```

NOT

```

if (blah == true) return;

```

DO THIS

```

if (blah == true)
{
    return;
}

```

AND THIS

```

for (int x = 0; x < y; x++)
{
    ...
}

```

- Avoid multiple statements per line for anything beyond simple get/set functionality.
- All header and source files have a unified look with common, non-ragged spacing.
- Put whitespace (a single space) after commas E.g. CallingFunction(blah, whatever, othervar)
- All math MUST have brackets to denote operation order. E.g. thisVar = ((20 * x) + 40) / 4.7f;

3. Files

A short description of each file's contents and purpose should be put at the top of the file

- Try to avoid files with multiple class definitions, although if necessary, group the classes together

- File names and class names should be closely related if not identical because it makes finding things far easier
- Global variables should be avoided unless there is a good reason for it.

4. Preprocessor features

- Conditional directives, `#if/#elif/#else`, must be followed by a compile time evaluated expression enclosed by braces

```
1 #if (cMyCiv == cCivZeus)
2 extern int gFoo = 0;
3 #endif
```

5. Rules

- Even though rules default to inactive, if you have an inactive rule always specifically define that it's inactive for readability.

6. Classes

- To overwrite the class method outside of the class mutable must be set on the class method.

```
1 class Foo
2 {
3     int a = 0;
4     int b = 0;
5     mutable int myMethod() { return(a + b); }
6 };
7
8 int Foo::myMethod()
9 {
10     return(a * b);
11 }
```

7. Notes about optimization

- Any expressions involving constant comparisons will be optimized away, including short circuit evaluations such as

```
1 if (constant1 == constant2 || expression)
```

AND

```
1 if (false && expression)
```

AND functions with only constant checks will be reduced, such as

```
1 bool civIsGreek()
2 {
3     if (cMyCiv == cCivZeus || cMyCiv == cCivHades || cMyCiv == cCivPoseidon)
4     {
5         return true;
6     }
7     return false;
```

```
8 }
```

will become

```
1 bool civIsGreek()  
2 { // True if cMyCiv is one of the Greek civs, or false when the if check is false  
3     return (true);  
4 }
```

AND any calls to such functions will be replaced with constants by the compiler to do the same optimization with constants.

- If expressions with value at compiled time evaluated true will have the check removed, on the other hand when the value is false the code block behind the check will be removed too. Same logic applies to switches and conditional operators
- Empty body void function calls including code being optimized away by compile time evaluation will also be removed
- Put code behind expressions which can be evaluated at compile time whenever possible, so the check and whole code block can be removed, e.g. any civ specific stuffs

8. Misc

- Use constants in switch cases even though XS allow variables or expressions

```
1 switch(variable1)  
2 {  
3     case constant1:  
4     {  
5         break;  
6     }  
7     case constant2:  
8     {  
9         break;  
10    }  
11 }
```

NOT

```
1 switch(variable1)  
2 {  
3     case variable2:  
4     {  
5         break;  
6     }  
7     case constant3:  
8     {  
9         break;  
10    }  
11 }
```